

ARK-498 — Networked Production-Like Performance Report

PRODUCTION-LIKE OVERHEAD CHARACTERIZATION · NOT A BENCHMARK CERTIFICATION · NOT A PRODUCTION SLA

Experiment: ARK-498

Preregistration: ARK-493-498-PREREGISTRATION-v1.1.md (§14), SHA-256 verified at run time

Decision: EXPERIMENT-PASS (all six hard criteria met)

Raw data: results/ark498_metrics.json

0. Reading this report honestly

The numbers below characterize the **overhead of ExecutionProof verification in a production-like networked environment**. They are **measured values with disclosed methodology**, not thresholds, not certifications, and **not a production SLA**. Every request paid a deliberately injected, constant dependency cost (policy lookup 50 ms + authority lookup 30 ms = 80 ms floor) plus real ProofRecord disk I/O. Latency is therefore dominated by the simulated dependencies by design; do **not** extrapolate these figures to production, and do **not** compare them to the prior in-process microsecond testbed (ARK-483-492) — different architecture, different scope.

1. Environment & methodology

Element	Value
Gate transport	Flask HTTP server on <code>127.0.0.1:5050</code> , real loopback TCP socket boundary
Client	Python <code>requests</code> over loopback
Policy-lookup latency (simulated)	50 ms (<code>time.sleep(0.050)</code>)
Authority-lookup latency (simulated)	30 ms (<code>time.sleep(0.030)</code>)
ProofRecord write	actual disk I/O (not mocked)
Latency clock	<code>time.monotonic()</code> , measured client-side end-to-end, reported in ms
Guard-B mode	deferred/batch, flushed at <code>/audit</code> ; signatures independently re-verified
Chain continuity	server resumed the ARK-493..497 hash chain via <code>load_tail()</code>
Total requests	~1,810 across 9 scenarios

Percentiles computed by linear interpolation over the sorted latency sample of each scenario.

Throughput computed over the **middle 60%** of request-completion timestamps (dropping the first and last 20%) to exclude ramp-up/drain.

2. Latency & throughput characterization (measured)

Metric	Scenario	Value
Cold-start latency	S001 (1 request)	85.2 ms
Warm p50	S002 (100 sequential)	83.9 ms
Warm mean	S002	83.9 ms
p50 under load	S003 (10 clients × 50 = 500)	84.4 ms
p95 under load	S003	88.0 ms
p99 under load	S003	93.9 ms
Sustained throughput (mid 60%)	S004 (5 clients × 200 = 1,000)	59.7 req/s
Error rate (server errors, not DENY)	S002 / S003	0.000 / 0.000

Interpretation. The ~84 ms median tracks the 80 ms injected dependency floor plus ~4 ms of gate + serialization + signing + hashing + loopback overhead. The tight p50→p99 spread (84→94 ms) under 10 concurrent clients indicates the verification path adds low and stable overhead **on top of** the dependency cost in this environment. Throughput (~60 req/s at 5 clients) is the arithmetic consequence of the 80 ms serial dependency floor under this concurrency, not a capacity ceiling of the gate.

3. Hard pass/fail criteria (FROZEN v1.1) — all met

Criterion	Requirement	Measured	Result
P-498-1 Fail-closed under dependency loss	leak count = 0 in S005/S006/S007	S005 leak 0 (fail-closed 20/20), S006 leak 0 (20/20), S007 leak 0 (20/20)	PASS
P-498-2 Zero duplicate executions	S009 duplicate executions = 0	5 unique keys × 10 sends = 50 requests → 5 executed entries, duplicates = 0	PASS
P-498-3 ProofRecord completeness	100% of accepted requests get one complete terminal record	1,766 / 1,766 complete = 100%	PASS
P-498-4 Error accounting	every error → HOLD/DENY (never ALLOW) with a ledger entry	server_errors = 0, ALLOW-on-error = 0	PASS
P-498-5 Recovery: no queued-denied auto-execute	0 automatic re-executions of denied requests after restoration	S008: 20 denied-window keys, 0 auto re-executions; recovery via new keys only	PASS
P-498-6 Signature verification	Guard-B verifies 100% of legitimate records	1,766 / 1,766 verified = 100%	PASS

3.1 Fail-closed detail (S005 / S006 / S007)

Each scenario ran 40 requests: 20 normal, then a sustained dependency failure injected at request 21.

Scenario	Failed dependency	Failure-window requests	ALLOW (leak)	DENY (fail-closed)
S005	policy lookup	20	0	20
S006	authority lookup	20	0	20
S007	ProofRecord store	20	0	20

Zero ALLOW decisions occurred while any named dependency was unavailable. When the store dependency is flagged unavailable, the gate resolves the would-be ALLOW to a fail-closed DENY and still

persists the DENY ProofRecord (so the decision remains fully accounted for under P-498-4) — no side effect executes.

3.2 Recovery detail (S008)

- 40 requests: requests 1–20 issued during an authority-lookup failure; dependency restored before request 21; requests 21–40 issued normally. Every request used a **new** idempotency key.
- **Recovery time (restoration → first successful ALLOW): 83.9 ms** — i.e. the first post-restoration request succeeded on its normal dependency path.
- First post-recovery ALLOW ProofRecord id: `979b04ac2bf11b868b7f3ca39e4d23a2`.
- Executed side-effects attributable to S008: **20** (exactly the 20 post-recovery requests). Denied-window requests were **not** auto-retried: **0** re-executions.

3.3 Duplicate-execution protection (S009)

- 5 clients, each submitting its own single idempotency key **10 times** concurrently (50 total requests, 5 unique keys).
- Executed side-effect ledger entries for S009: **5** (exactly one per unique key). Duplicate executions beyond first: **0**.
- Per-key serialization is enforced by a per-idempotency-key lock at the enforcement point; distinct keys proceed fully in parallel.

4. Independent verification audit (all ARK-498 records)

Audit metric	Value
ARK-498 ProofRecords produced	1,766
Unique ProofRecord ids	1,766
Complete records (all required fields present)	1,766 (100%)
Guard-B independent signature/verification PASS	1,766 (100%)
Guard-B isolated-import analysis	<code>permitted_only = true</code> (no gate / enforcement / tools / guards / actor imports)

5. Threats to validity / disclosures

1. **Dependency latencies are simulated constants**, not real network/database round-trips. Real deployments will differ.
2. **Loopback TCP** removes real network variance (NIC, switch, TLS). A real socket boundary is crossed, but not a real network.
3. **Single host**, testbed `ed25519` key derived from a fixed seed (`b'\x00'*32`) — a testbed key, never a production secret.

4. Throughput reflects the injected 80 ms serial floor at the tested concurrency, **not** the gate's maximum capacity.
5. These results **do not** transfer to production and **do not** supersede or compare to ARK-483-492.

Bottom line: In a production-like networked configuration, ExecutionProof verification held every safety property under load and dependency failure (fail-closed, no duplicate execution, complete and independently-verifiable proofs, clean recovery) while adding low, stable overhead **on top of** the simulated dependency cost. The latency/throughput figures are honest characterization data and are explicitly not a benchmark certification or a production SLA.